Lab 04

Introduction to Classes & Objects

Objectives: ***At the end of this lab session the student should be able to:***

1. Declare a class and use it to create an object.
2. Declare methods in a class to implement the class's behaviors.
3. Declare instance variables (attributes) in a class.
4. Call an object's method to make that method perform its task.
5. Learn the differences between instance variables of a class and local variables of a method.
6. Use a constructor and initialize objects with constructors.
7. Distinguish between primitive types and reference types.

i

Part 1: Problem Solving Exercises (*Instructor-led*) (Time: 90 minutes)

Exercise 1:

a) Create a new project, call it "bank", and then add to it the following class:

```
public class Account {
        private String name; //instance variable

        public String getName() {
                return name;
        }

        public void setName(String name) {
                this.name = name;
        }
}
```

b) Create a new class called "AccountTest" that contains a main method. In the main method, create an object of class Account, and print the value of variable "name" of this object.

c) Do we need to use import statement to import "Account" class?
Yes [ ] No [ ] Why?_____

d) Is it necessary to use word *this* in method *setName*?

e) What are the default values for instance variables of the following types

int	
double & float	
boolean	
Object	

f) Add a constructor that specifies custom initialization for the instance variable "name", now try to run AccountTest again

g) Which of the following statements is correct and which is false:

- If a class does not define constructors, the compiler provides a default constructor with no parameters.
- There is no default constructor in a class that declares constructor.
- The class can have only one constructor.

h) Add another instance variable called “balance”, and add setter/getter methods(use Eclipse facility to add set/get methods). Set method should ensure that balance won’t be negative.

i) Add another constructor that specify custom initialization for variables: name, balance. The constructor should ensure that balance won’t be negative.

j) Add a method called *deposit* that increase the balance, the method should ensure that deposit *amount* is greater than zero.

k) Add a method called *withdraw* that withdraws money from an account. Ensure that withdrawal amount does not exceed the account’s balance. If it does, the balance should be left unchanged and the method should print a message indicating “ withdrawal amount exceeded account balance”.

l) Test these methods in the class AccountTest.

Exercise 2: Debugging

The program in this section does not compile. Fix all the compilation errors so that the program will compile successfully. Once the program compiles, execute the program, and compare its output with the sample output; then eliminate any logic errors that may exist. The sample output demonstrates what the program’s output should be once the program’s code is corrected.

Sample Output

```
Created Ahmed Ibrahim, age 19
Happy Birthday to Ahmed Ibrahim
```

Broken code to be fixed:

```
// Person.java
// Creates and manipulates a person with a first name, last name and age

public class Person {
  private String firstName;
  private String lastName;
  private int age;

  public void Person( String first, String last, int years ) {
    firstName = first;
    lastName = last;

    if ( years < 0 )
      age = years;
  } // end Person constructor

  public String getFirstName( String FirstName ) {
    return firstName;
  } // end method getFirstName
```

```
   public setFirstName( String first ) {
      firstName = first;
   } // end method setFirstName

   public String getLastName() {
      return;
   } // end method getLastName

   public void setLastName( String last ) {
      lastName = last;
   } // end method setLastName

   public int getAge() {
      return years;
   } // end method getAge

   public void setAge( int years ) {
      if ( years > 0 )
         age = years;
   } // end method setAge
} // end class Person


/**************************************************************************
 * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
 * Pearson Education, Inc. All Rights Reserved.                           *
 **************************************************************************/

// PersonTest.java
// Test application for the Person class

public class PersonTest {
   public static void main( String args[] ) {
      Person person = Person( "Ahmed", "Ibrahim", 19 );

      System.out.printf( "Created %s %s, age %d\n",
         getFirstName(), getLastName(), getAge() );

      person.setAge = person.getAge() + 1;
      System.out.printf( "Happy Birthday to %s %s\n",
         person.getFirstName(), person.getLastName() );
   } // end main
} // end class PersonTest

/**************************************************************************
 * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
 * Pearson Education, Inc. All Rights Reserved.                           *
 **************************************************************************/
```

Exercise 3: Constructor Overloading

```
// Person.java
 public class Person {
   private String firstName;
   private String lastName;
   private int age;

   public Person( String firstName, String lastName, int age ) {
     setFirstName(firstName);
     setLastName(lastName);
     setAge(age);
   } // end Person constructor

   public String getFirstName() {
     return firstName;
   } // end method getFirstName

   public void setFirstName( String first ) {
     firstName = first;
   } // end method setFirstName

   public String getLastName() {
     return lastName;
   } // end method getLastName

   public void setLastName( String last ) {
     lastName = last;
   } // end method setLastName

   public int getAge() {
     return age;
   } // end method getAge

   public void setAge( int years ) {
     if ( years > 0 )
       age = years;
   } // end method setAge
} // end class Person
/**************************************************************************
 * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
 * Pearson Education, Inc. All Rights Reserved.                           *
 **************************************************************************/


// PersonTest.java
// Test application for the Person class

public class PersonTest {
   public static void main( String args[] ) {
     Person person = new Person( "Ahmed", "Ibrahim", 19 );

     System.out.printf( "Created %s %s, age %d\n",
       person.getFirstName(), person.getLastName(), person.getAge() );

     person.setAge( person.getAge() + 1 );
```

```
      System.out.printf( "Happy Birthday to %s %s\nYour age is now %d\n",
        person.getFirstName(), person.getLastName(), person.getAge() );
   } // end main
} // end class PersonTest

/**************************************************************************
 * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
 * Pearson Education, Inc. All Rights Reserved.                           *
 **************************************************************************/
```

- Run the PersonTest.java and make sure that your output is similar to:

  Created Ahmed Ibrahim, age 19
  Happy Birthday to Ahmed Ibrahim
  Your age is now 20

- Now add to the Person class, two double attributes representing the person weight and height. Generate setters and getters for both fields.
- Add a five-parameter constructor that initializes all the attributes to the values passed as parameters. Let your new constructor call the existing constructor to initialize the first three attributes [hint: use *this()*]. Now the class has two constructors: what is the name of the OOP concept that allows more than one method to have the same name? _____ _____
- Add an instance method called *getBMI()* to return the BMI (body-mass index) for a Person. The formula for BMI is $MI = \frac{weight}{height^2}$. Make sure that the method works only for positive values of weight and height, otherwise it returns 0.0.

In the ***PersonTest*** class, create a new Person instance using the 5-parameter constructor and your own information. Let the program print the new person details, together with the BMI value. Print a message saying "Overweight" if the BMI for the person is more than or equal 25, otherwise print "Healthy".

Part 2: Practice Exercises (Time: 45 minutes)

Exercise 4: GradeBook Application

- Create a new Java Project.
- Add the following two classes to your project **src** folder:

```
// Lab 4: GradeBook.java
// GradeBook class with a constructor to initialize the course name.

public class GradeBook {
   private String courseName; // course name for this GradeBook
   /* write code to declare a second String instance variable */

   // constructor initializes courseName with String supplied as argument
   public GradeBook( String name ) {
      courseName = name; // initializes courseName
   } // end constructor

   // method to set the course name
   public void setCourseName( String name ) {
      courseName = name; // store the course name
   } // end method setCourseName
```

```
   // method to retrieve the course name
   public String getCourseName() {
      return courseName;
   } // end method getCourseName

   /* write code to declare a get and a set method for the instructor's name */

   // display a welcome message to the GradeBook user
   public void displayMessage() {
      // this statement calls getCourseName to get the
      // name of the course this GradeBook represents
      System.out.printf( "Welcome to the grade book for\n%s!\n", getCourseName() );
      /* write code to output the instructor's name */
   } // end method displayMessage

} // end class GradeBook
/**************************************************************************
 * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
 * Pearson Education, Inc. All Rights Reserved.                           *
 **************************************************************************/

// Lab 4: GradeBookTest.java
// GradeBook constructor used to specify the course name at the
// time each GradeBook object is created.

public class GradeBookTest {
   // main method begins program execution
   public static void main( String args[] ) {
      // create GradeBook object
      GradeBook gradeBook1 = new GradeBook("CMPS252 OO Programming Lab" );

      gradeBook1.displayMessage(); // display welcome message

      /* write code to change instructor's name and output changes */

   } // end main
} // end class GradeBookTest
/**************************************************************************
 * (C) Copyright 1992-2012 by Deitel & Associates, Inc. and               *
 * Pearson Education, Inc. All Rights Reserved.                           *
 **************************************************************************/
```

- Run the **GradeBookTest** application to verify that your classes are copied and work correctly.
- Perform the following on the GradeBook class:
  1. Declare a String *instance variable* to represent the *instructor's name*.
  2. Declare a public *set method* for the instructor's name that *does not return a value* and *takes a String as a parameter*. In the body of the set method, assign the parameter's value to the variable that represents the instructor's name.
  3. Declare a public *get method* that returns a String and takes no parameters. This method should return the instructor's name.
  4. Modify the constructor to take two String parameters. Assign the parameter that represents the instructor's name to the appropriate instance variable.
  5. Add a System.out.printf statement to method *displayMessage()* to output the value of the instance variable you declared earlier.

- Perform the following on the GradeBookTest class:

1. Add your name as the instructor for the CMPS252 course and display.
2. Immediately after the call to displayMessage(), output a message saying: “Chaning course instructor:”
3. Using a setter, set the course instructor to be your friend’s name.
4. Call the displayMessage() again to refelct the changes.
5. Create a new GradeBook object for the course CMPS251: OO Programming and make the instructor the actual current instructor of your section. Display the new course information.

Additional Practice Problems

Exercise 5: Employee Application

Create a class called **Employee** that includes three pieces of information as instance variables—a first name (type String), a last name (type String) and a monthly salary (type double). Your class should have a constructor that initializes the three instance variables. Provide a *set* and a *get* method for each instance variable. If the monthly salary is not positive, set it to 0.0. Write a test application named **EmployeeTest** that demonstrates class Employee’s capabilities. Create two Employee objects and display the yearly salary for each Employee. Then give each Employee a 10% raise and display each Employee’s yearly salary again.

Sample Output:

```
Employee 1: Bob Jones; Yearly Salary: 34500.00
Employee 2: Susan Baker; Yearly Salary: 37809.00

Increasing employee salaries by 10%
Employee 1: Bob Jones; Yearly Salary: 37950.00
Employee 2: Susan Baker; Yearly Salary: 41589.90
```

Problem Solving Tips:

a. Class Employee should declare three instance variables.

b. The constructor must declare three parameters, one for each instance variable. The value for the salary should be validated to ensure it is not negative.

c. Declare a public *set* and *get* method for each instance variable. The *set* methods should not return values and should each specify a parameter of a type that matches the corresponding instance variable (String for first name and last name, double for the salary). The *get* methods should receive no parameters and should specify a return type that matches the corresponding instance variable.

d. When you call the constructor from the test class EmployeeTest, you must pass it three arguments that match the parameters declared by the constructor.

e. Giving each employee a raise will require a call to the *get* method for the salary to obtain the current salary and a call to the *set* method for the salary to specify the new salary.

A salary is a dollar amount, so you should output the salary using the %.2f specifier to provide two digits of precision.

Exercise 6: Creating an Invoice Class

Create a class called Invoice that a hardware store might use to represent an invoice for an item sold at the store. An Invoice should include four pieces of information as instance variables—a part number (type String), a part description (type String), a quantity of the item being purchased (type int) and a price per item (double). Your class should have a constructor that initializes the four instance variables. Provide a set and a get method for each instance variable. In addition, provide a method named getInvoiceAmount that calculates the invoice amount (i.e. multiplies the quantity by the price per item), then returns the amount as a double value. If the quantity is not positive, it should be set to 0. If the price per item is not positive, it should be set to 0.0. Write a test application named InvoiceTest that demonstrates class Invoice's capabilities.